
DSC 40B - Discussion 01

Problem 1.

What is the time complexity of the following functions? State your answer using Θ notation.

a)

```
def foo(n):  
    for i in range(n**2 - 2*n + 100):  
        j = 0  
        while j < n:  
            j += 1
```

Solution: $\Theta(n^3)$

b)

```
def foo(n):  
    while n > 1:  
        n /= 10  
        print(n)
```

Solution: $\Theta(\log n)$

c)

```
def foo(n):  
    for i in range(n):  
        for j in range(i**2): # <-- notice the bound!  
            print(i + j)
```

Solution: $\Theta(n^3)$

Problem 2.

Let $f(n) = \sum_{p=0}^n 3^p$. What is f in Θ notation?

Solution:

General form of a geometric sum $\sum_{p=0}^n x^p = \frac{1 - x^{n+1}}{1 - x}$.

Substituting our equation yields $\sum_{p=0}^n 3^p = \frac{1 - 3^{n+1}}{1 - 3}$.

Therefore, $f(n) = \Theta(3^n)$ after throwing out the constants.

Problem 3.

Consider the algorithm below.

```
def bogosearch(numbers, target):  
    """search by randomly guessing. `numbers` is an array of n numbers"""  
    n = len(numbers)
```

```

while True:
    # randomly choose a number between 0 and n-1 in constant time
    guess = np.random.randint(n)
    if numbers[guess] == target:
        return guess

```

We will set up the analysis of the expected time complexity of this algorithm.

- a) What are the cases? How many are there?

Solution: Case α occurs when the target is found on iteration α . In principle, the algorithm can run forever, although this is very unlikely (it happens with probability zero). As such, there are infinitely-many cases.

- b) What is the probability of case α ?

Solution: $P(\alpha)$ is the probability of guessing wrong $\alpha - 1$ times and guessing right on the α th time:

$$(1 - 1/n)^{\alpha-1} \cdot (1/n)$$

- c) What is the running time in case α ?

Solution: We perform α iterations in case α , each taking constant time, c . The total work in case α is therefore $c\alpha$.

Problem 4.

For each problem below, state the largest theoretical lower bound that you can think of and justify (that is, your bound should be “tight”). Provide justification for this lower bound. You do not need to find an algorithm that satisfies this lower bound.

Example: Given an array of size n and a target t , determine the index of t in the array.

Example Solution: $\Omega(n)$, because in the worst case any algorithm must look through all n numbers to verify that the target is not one of them, taking $\Omega(n)$ time.

- a) Given an array of n numbers, find the second largest number.

Solution: $\Omega(n)$.

Any algorithm must look through all n numbers to be sure it didn’t miss the second largest number, and so must take $\Omega(n)$ time. Algorithm: loop through the numbers, keeping track of the maximum and the next largest number seen.

- b) Given an array of n numbers, check to see if there are any duplicates.

Solution: $\Omega(n)$

Again, we have to look through all of the numbers in the worst case; if we don’t, we can’t be sure that it isn’t a duplicate of a number we have looked at.

It turns out that the actual, *tight* theoretical lower bound for this problem is $\Omega(n \log n)$, under some assumptions.^a But justifying that lower bound is not trivial, and this question asked you to state the best TLB that you could justify.

If you said that the TLB is $\Omega(n \log n)$ because you thought of sorting the list and checking for consecutive equal elements, this is good thinking – but in order to claim that the TLB is therefore

$\Omega(n \log n)$, you'd also need to justify the assumption that sorting (or something like it) is *necessary* of any optimal algorithm. That's not an easy claim to justify!

^aSee: https://en.wikipedia.org/wiki/Element_distinctness_problem

- c) Given an array of n integers (with $n \geq 2$), check to see if there is a pair of elements that add to be an even number.

Solution: $\Omega(1)$

This can be done in constant time by checking only the first 2 numbers. If the first two are both even, or both odd, then the answer is “yes”, since the sum will then be even. If the first two numbers are of different polarity – one even and the other odd – then the answer is “no” if the size of the list is 2, otherwise it is “yes” since the third number in the array must be either even or odd; pairing it with one of the first two numbers with the same polarity gives an even sum.

Problem 5.

Consider the code below where heights is an array of n elements:

```
for i in range(n):
    for j in range(2*i):
        height = heights[i] + heights[j]
```

What is the time complexity of the code?

Solution:

We can see that outer iteration runs n times, for inner iteration:

On outer iter 1, inner body runs 0 times

On outer iter 2, inner body runs 2 times

On outer iter 3, inner body runs 4 times

Hence,

On outer iter α , inner body runs $(2\alpha - 2)$ times

$$\sum_{\alpha=1}^n f(\alpha) = \sum_{\alpha=1}^n 2 * \alpha - 2$$

$$\sum_{\alpha=1}^n 2 * \alpha - 2 = 0 + 2 + 4 + \dots + (2n - 4) + (2n - 2)$$

This is an arithmetic series and we know the formula for sum of an arithmetic series is:

$$Sum = \frac{n}{2} * (a_1 + a_n)$$

Where n is the number of terms, a_1 is the first term in the series and a_n is the last term. Therefore sum of the series:

$$Sum = \frac{n}{2} * (0 + 2n - 2) = n(n - 1) = n^2 - n$$

Therefore the time complexity can be given as $\theta(n^2)$.